

9.Reversed-Z

统一各个平台的Z值

1. Reversed_Z的平台
2. 把近裁剪面从[n,0]映射到[0,far]
3. ProjectionParams
4. 投影矩阵

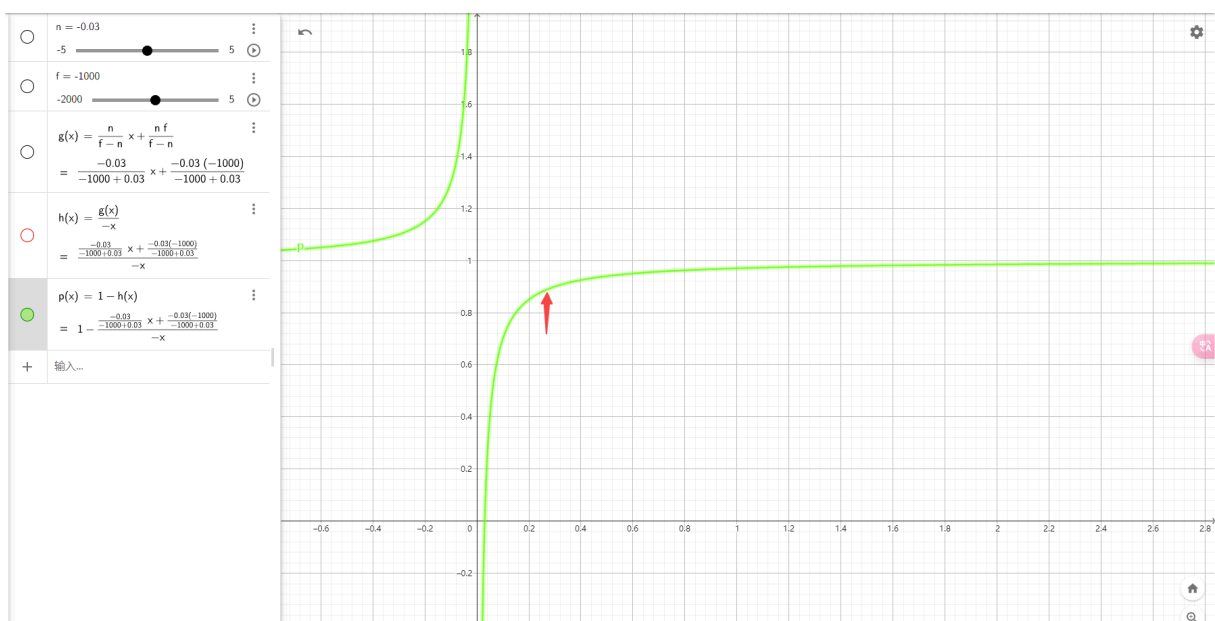
NDC下的Z值，就是我们通常说的深度

按照OpenGL的传统，这个立方体的x、y、z分量的范围都是[-1, 1]。

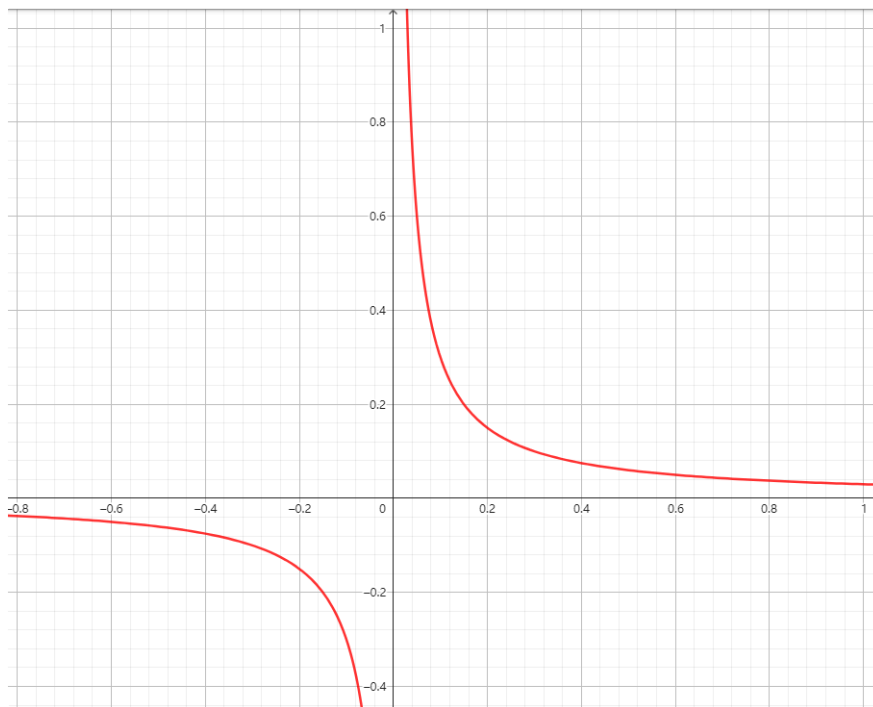
但在DirectX这样的API中，z分量的范围本来是[0, 1]，但是有一个Reversed-Z的问题，所以是[1,0]

在这个模式下，可以看到 $Z < 0.3$ 占据了大概有80%的空间

当摄像机离的越来越远的时候，非常多的距离共用了一个深度值，容易导致ZFighting的问题



没有Reversed-Z的情况



Reversed-Z的情况

浮点值的问题

假设 $a=1.2$

小数部分和整数部分会分别存储

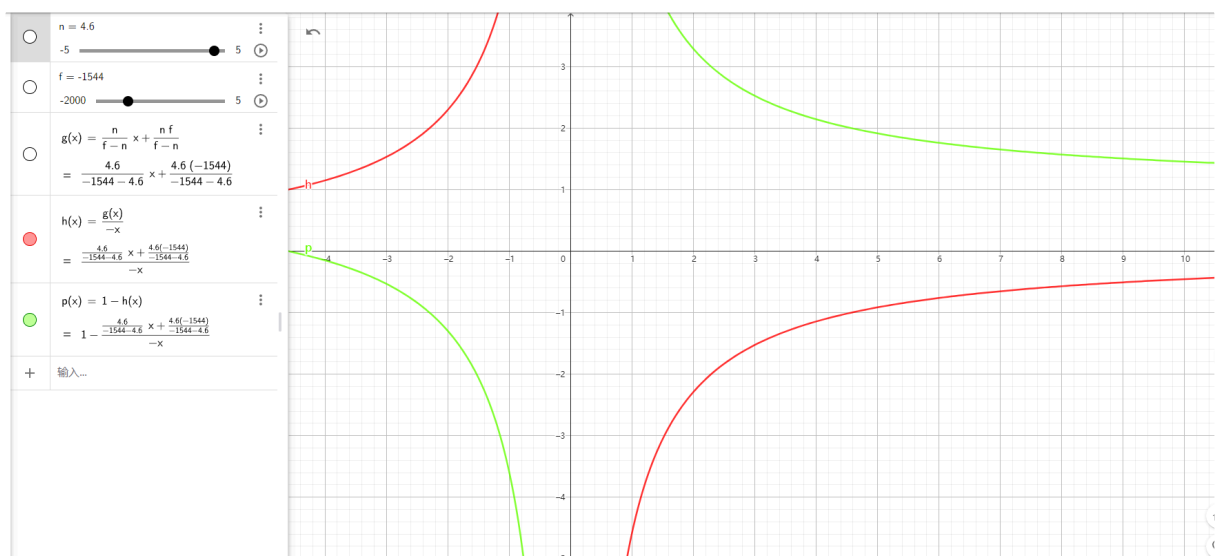
越接近0的位置，精度越高；大于0，精度逐渐降低

从曲线上，翻转以后，远处映射到了接近0的地方

这样保证了最大化的利用了浮点数的精度

引入了Reversed-Z以后，有时候依然无法避免ZFighting的问题

可以把近裁剪面改大一点，远裁剪面改小一点，这样曲线就没有那么陡峭，存储的值就更多一些



统一各个平台的Z值

在unity中，有一个函数UNITY_Z_0_FAR_FROM_CLIPSPACE

把裁剪空间的Z值变换到0到Far

代码如下

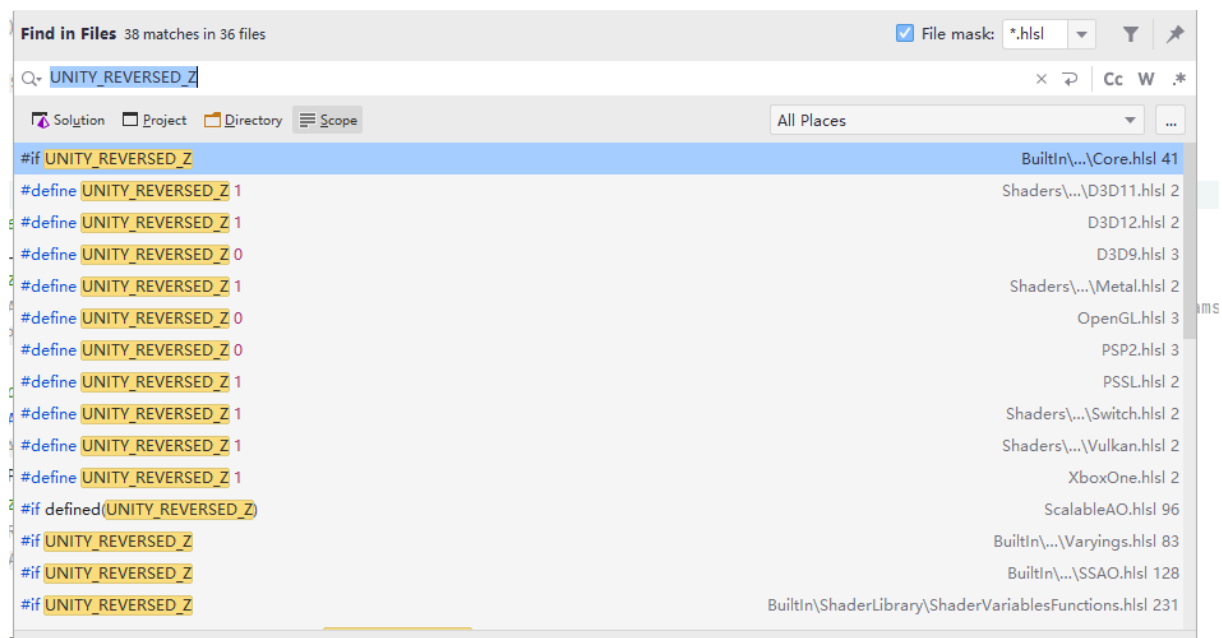
UNITY_UV_STARTS_AT_TOP和UNITY_REVERSED_Z几乎一样

DX类的平台为真

Opengl类的为假

```
1 //UNITY_REVERSED_Z在D3D为1, 在OpenGL为0
2 #if UNITY_REVERSED_Z
3
4     // TODO: workaround. There's a bug where SHADER_API_GL_CORE gets erroneously defined on switch.
5     #if (defined(SHADER_API_GLCORE) && !defined(SHADER_API_SWITCH)) || defined(SHADER_API_GLES) || defined(SHADER_API_GLES3)
6         //GL with reversed z => z clip range is [near, -far] -> remapping to [0, far]
7         //一般不会走到这里
8         #define UNITY_Z_0_FAR_FROM_CLIPSPACE(coord) max((coord - _ProjectionParams.y)/(-_ProjectionParams.z - _ProjectionParams.y)*_ProjectionParams.z, 0)
9     #else
10        //参考注释2
11        //D3d with reversed Z => z clip range is [near, 0] -> remapping to [0, far]
12        //max is required to protect ourselves from near plane not being correct/meaningful in case of oblique matrices.
13        #define UNITY_Z_0_FAR_FROM_CLIPSPACE(coord) max(((1.0-(coord)/_ProjectionParams.y)*_ProjectionParams.z),0)
14    #endif
15 #elif UNITY_UV_STARTS_AT_TOP
16     //D3d without reversed z => z clip range is [0, far] -> nothing to do
17     #define UNITY_Z_0_FAR_FROM_CLIPSPACE(coord) (coord)
18 #else
19     //Opengl => z clip range is [-near, far] -> remapping to [0, far]
20     #define UNITY_Z_0_FAR_FROM_CLIPSPACE(coord) max(((coord + _ProjectionParams.y)/(_ProjectionParams.z + _ProjectionParams.y))*_ProjectionParams.z, 0)
21 #endif
```

1. Reversed_Z的平台



2. 把近裁剪面从[n,0]映射到[0,far]

$$[n \sim 0] \gg [0 \sim \text{far}]$$

$$n < x < 0$$

$$n - n < x - n < 0 - n$$

$$0 < x - n < -n$$

$$0 < (x - n) \frac{1}{-n} < -n \frac{1}{-n}$$

$$0 < -\frac{x - n}{n} < 1$$

$$0 < -\frac{x - n}{n} f < f$$

$$0 < \frac{n - x}{n} f < f$$

$$0 < (\frac{n}{n} - \frac{x}{n}) f < f$$

$$0 < (1 - \frac{x}{n}) f < f$$

3. ProjectionParams

```
9-DissolveRect.shader x Core.hlsl x UnityInput.hlsl x D3D11.hlsl x ShaderVariablesFunctions.hlsl x
38
39 // Time (t = time since current level load) values from Unity
40 float4 _Time; // (t/20, t, t*2, t*3)
41 float4 _SinTime; // sin(t/8), sin(t/4), sin(t/2), sin(t)
42 float4 _CosTime; // cos(t/8), cos(t/4), cos(t/2), cos(t)
43 float4 _unity_DeltaTime; // dt, 1/dt, smoothdt, 1/smoothdt
44 float4 _TimeParameters; // t, sin(t), cos(t)
45
46 #if !defined(USING_STEREO_MATRICES)
47 float3 _WorldSpaceCameraPos;
48 #endif #if !defined(USING_STEREO_MATRICES)
49
50 // x = 1 or -1 (-1 if projection is flipped)
51 // y = near plane
52 // z = far plane
53 // w = 1/far plane
54 float4 _ProjectionParams;
55
56 // x = width
57 // y = height
58 // z = 1 + 1.0/width
59 // w = 1 + 1.0/height
60 float4 _ScreenParams;
61
62 // Values used to linearize the Z buffer (http://www.humus.name/temp/Linearize%20depth.txt)
63 // x = 1-far/near
64 // y = far/near
65 // z = x/far
66 // w = y/far
67 // or in case of a reversed depth buffer (UNITY_REVERSED_Z is 1)
68 // x = -1+far/near
69 // y = 1
70 // z = x/far
71 // w = 1/far
72 float4 _ZBufferParams;
73
```

4. 投影矩阵

投影矩阵

如果处于深度 (Z) 发生反转的平台上, 则 `GL_GetGPUProjectionMatrix()` 返回一个还原了 z 的矩阵。但是, 如果要手动从投影矩阵中进行合成 (例如, 对于自定义阴影或深度渲染), 您需要通过脚本按需自行还原深度 (Z) 方向。

以下是执行此操作的示例:

```
var shadowProjection = Matrix4x4.Orthonormal(...); //阴影摄像机投影矩阵
var shadowViewMat = ... //阴影摄像机视图矩阵
var shadowSpaceMatrix = ... //从裁剪空间到阴影贴图纹理空间

//当引擎通过摄像机投影计算设备投影矩阵时,
//“m_shadowCamera.projectionMatrix”被隐式反转
m_shadowCamera.projectionMatrix = shadowProjection;

//“shadowProjection”在连接到“m_shadowMatrix”之前被手动翻转,
//因为它被视为着色器的其他矩阵。
if(SystemInfo.usesReversedZBuffer)
{
    shadowProjection[2, 0] = -shadowProjection[2, 0];
    shadowProjection[2, 1] = -shadowProjection[2, 1];
    shadowProjection[2, 2] = -shadowProjection[2, 2];
    shadowProjection[2, 3] = -shadowProjection[2, 3];
}

m_shadowMatrix = shadowSpaceMatrix * shadowProjection * shadowViewMat;
```